Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 02

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

Identificar y formalizar los requisitos de una problemática real (tienda, biblioteca, reservas, etc.) como base para el diseño de una aplicación Django.

Diseñar e implementar una nueva App Django (Model, View, URL, Template) conectada a un proyecto existente, aplicando el patrón MVT.

Implementar formularios (Forms) que traduzcan los requisitos identificados en una funcionalidad real de captura y persistencia de datos.

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

Introducción al laboratorio

En las sesiones anteriores construimos y extendimos el proyecto DjangoInicial, comprendiendo cómo Django organiza una aplicación mediante el patrón MVT: Project, Apps, Models, Views, URLs y Templates. En este laboratorio damos un paso más cercano a cómo se trabaja en un proyecto real: en lugar de partir de un modelo ya definido, ustedes van a investigar una problemática real, capturar sus requisitos, y traducir esos requisitos en una aplicación Django funcional, construida como una nueva App conectada al proyecto que ya conocen (config/core).

Para mantener el foco en el flujo MVT y en cómo un formulario transforma datos en un response, esta aplicación no usará base de datos: los datos se definirán de forma estática dentro de models.py (por ejemplo, como una lista de diccionarios), y las vistas leerán y agregarán información directamente sobre esa estructura en memoria, sin migraciones ni panel de administración.

Pueden elegir cualquier problemática cotidiana que se preste a una aplicación sencilla: tienda de barrio, biblioteca, veterinaria, alquiler de equipos, reserva de citas, control de asistencia, u otra que les resulte relevante.

Enunciados

Ejercicio 1 — Investigar una problemática real
Investiguen una problemática cotidiana que pueda resolverse con una aplicación web (por ejemplo: tienda de barrio, biblioteca, veterinaria, alquiler de equipos, reserva de citas, control de asistencia, u otra de su interés). Redacten en 3-5 líneas en qué consiste el problema y quién lo usaría.

Ejercicio 2 — Capturar los requisitos
A partir de la problemática elegida, listen entre 4 y 6 requisitos funcionales del sistema (ejemplo: "el sistema debe permitir registrar un producto", "el usuario debe poder ver el listado de productos disponibles"). Cada requisito debe redactarse como una acción concreta que el sistema debe permitir.

Ejercicio 3 — Diseñar el modelo de datos
A partir de los requisitos del Ejercicio 2, identifiquen la entidad principal de su problemática (por ejemplo: Producto, Libro, Cita, Equipo) y definan sus campos (nombre, tipo de dato y si es obligatorio). Justifiquen brevemente por qué cada campo es necesario según los requisitos capturados.

Ejercicio 4 — Crear la nueva App
Dentro del proyecto base (config/core), creen una nueva App con un nombre representativo de su problemática (por ejemplo store, library, bookings). Regístrenla en INSTALLED_APPS.

Ejercicio 5 — Implementar el Model con datos estáticos
Traduzcan el diseño del Ejercicio 3 en models.py, definiendo los datos como una lista de diccionarios (o de objetos de una clase simple) con al menos 5 registros de ejemplo. No se usan migraciones ni base de datos: esta lista es la única fuente de datos de la aplicación.

Ejercicio 6 — Implementar el listado (View + URL + Template)
Creen una vista que recorra la lista definida en models.py y la muestre en un template que herede de base.html, siguiendo el mismo patrón usado en core.

Ejercicio 7 — Implementar el formulario (Forms)
Creen un forms.py con un forms.Form (no ModelForm, ya que no hay modelo de base de datos) cuyos campos correspondan a los requisitos capturados en el Ejercicio 2.

Ejercicio 8 — Implementar la vista de creación
Creen una vista que muestre el formulario del Ejercicio 7, procese el envío (POST), valide los datos y agregue el nuevo registro a la lista en memoria definida en models.py. Luego redirijan al listado para confirmar que el nuevo dato aparece. Tengan en cuenta que, al no haber base de datos, los datos agregados se pierden al reiniciar el servidor — esto es esperado y debe mencionarse en su documentación.

Ejercicio 9 — Verificación del flujo completo
Naveguen su aplicación de principio a fin (listado → crear → volver al listado con el nuevo dato reflejado) y documenten con capturas el recorrido Request → URL → View → Model (datos estáticos) → Template → Response. Expliquen cómo su nueva App convive con core dentro del mismo Project.

Ejercicio 10 — Publicar en GitHub

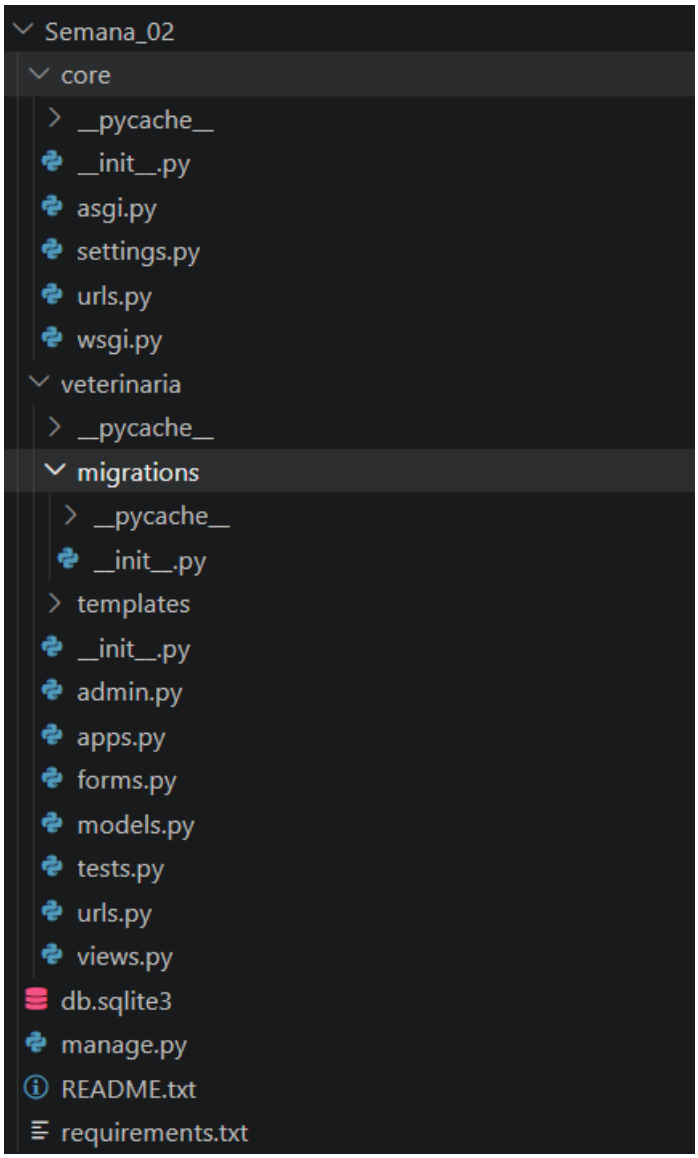
Actualicen requirements.txt y README.md describiendo su problemática, sus requisitos y la App creada. Hagan commit y push al repositorio del equipo.

Entregables

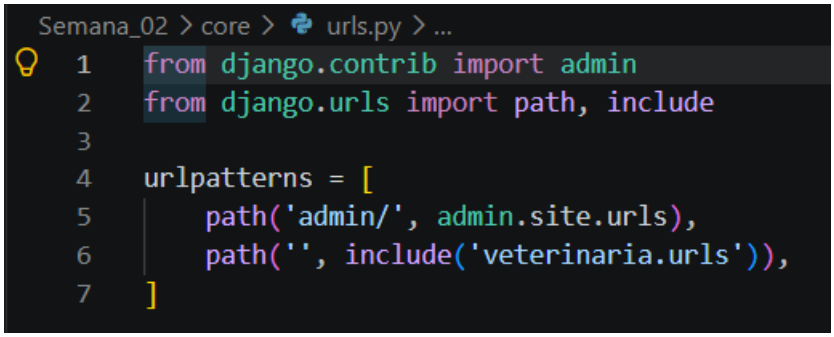
- Documento de requisitos: problemática elegida (Ejercicio 1) y lista de requisitos funcionales (Ejercicio 2).
- Diseño del modelo de datos: entidad principal, campos, tipos y justificación (Ejercicio 3).
- Código fuente completo de la nueva App: models.py (con los datos estáticos), views.py, urls.py, forms.py y templates (listado y formulario).
- Capturas de pantalla del flujo funcionando: listado de registros, formulario de creación, y el nuevo registro reflejado en el listado.
- Explicación del flujo MVT aplicado a su caso (Ejercicio 9), incluyendo cómo su App se conecta con core dentro del mismo Project.
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo, según el formato de evidencia indicado en las instrucciones del laboratorio (nombre, título, captura, código, explicación y casos de prueba).

EVIDENCIAS:

ESTRUCTURA:



CODIGO:



Semana_02 > veterinaria > apps.py > ...

```python
from django.apps import AppConfig


class VeterinariaConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'veterinaria'

```

Semana_02 > veterinaria > forms.py > ...

```python
from django import forms

class MascotaForm(forms.Form):
    nombre = forms.CharField(label="Nombre de la mascota", max_length=50)
    especie = forms.CharField(label="Especie", max_length=50)
    raza = forms.CharField(label="Raza", max_length=50)
    peso = forms.DecimalField(label="Peso (kg)", max_digits=5, decimal_places=2)
```

Semana_02 > veterinaria > models.py > ...

```python
from django.db import models

# Create your models here.

mascotas = [
    {'nombre': 'Tigger', 'especie': 'Gato', 'raza': 'Atigrado Naranja', 'peso': 4.40},
    {'nombre': 'Mushu', 'especie': 'Gato', 'raza': 'Angorita', 'peso': 5.50},
    {'nombre': 'Bobby', 'especie': 'Perro', 'raza': 'Pastor Alemán', 'peso': 19.00},
    {'nombre': 'Luna', 'especie': 'Gato', 'raza': 'Siamés', 'peso': 3.10},
    {'nombre': 'Max', 'especie': 'Perro', 'raza': 'Labrador', 'peso': 25.30}
]
```

```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.listar_mascotas, name='listar_mascotas'),
    path('crear/', views.crear_mascota, name='crear_mascota'),
]
```

Semana_02 > veterinaria > views.py > ...

```python
from django.shortcuts import render

# Create your views here.

from django.shortcuts import render, redirect
from .models import mascotas
from .forms import MascotaForm

def listar_mascotas(request):
    return render(request, 'veterinaria/listar.html', {'mascotas': mascotas})

def crear_mascota(request):
    if request.method == 'POST':
        form = MascotaForm(request.POST)
        if form.is_valid():
            mascotas.append(form.cleaned_data)
            return redirect('listar_mascotas')
    else:
        form = MascotaForm()
    return render(request, 'veterinaria/crear.html', {'form': form})
```

Conclusiones:

Comprensión del patrón MVT: En este laboratorio me enfoqué en dominar el ciclo de vida de una petición web bajo el patrón Modelo-Vista-Template (MVT). Logré estructurar la aplicación clínica veterinaria organizando el enrutamiento (urls.py), los controladores o funciones de vista (views.py) y las plantillas visuales (templates).

Manejo de estructuras en memoria: A diferencia de la semana anterior, el reto fue comprobar cómo fluye la información sin interactuar directamente con una base de datos. Trabajé con colecciones de datos simulados (listas de diccionarios con pacientes, doctores y servicios) definidos en el propio archivo de vistas. Aprendí que, aunque guardar datos en la memoria RAM del servidor sirve para probar interfaces rápidamente, tiene la gran limitación de que cualquier información se pierde de inmediato al detener o reiniciar el servidor.

Renderizado dinámico en el frontend: Consolidé el uso de las etiquetas del motor de plantillas de Django (DTL). Aprendí a diferenciar el uso de la lógica mediante llaves con porcentaje ({% for %} y {% if %}) para iterar colecciones y aplicar condiciones, frente al uso de dobles llaves ({{ variable }}) para imprimir valores específicos en el navegador. Con esto logré una navegación web modular y un despliegue limpio de información clínica en pantalla.